

① Features of JUnit Test Cases

- i] annotations : Uses markers like `@Test` to identify methods as test cases.
- ii] Assertions : provides methods to compare expected result with actual outcome.
- iii] Test runners : allow for the automated execution of test suites.
- iv] Fixtures : Support `@Before` and `@After` to set up / clean up the environment for each test.
- v] Speed : executes tests rapidly as they do not require a full system deployment.

② JUnit Classes

- i] Assert class : contains static methods to check if a condition is true or false (e.g., `assertEquals`).
- ii] TestCase class : a class that defines the fixture to run multiple tests (primary used in JUnit 4).
- iii] TestSuite class : a container used to group multiple test cases together for batch execution.
- iv] TestResult classes : collects the result of all tests and identifies failures.

③ Importance API's of JUnit.

- `assertEquals (expected, actual)`

- assertTrue (condition)
- assertEquals (expectedArray, actualArray).
- @BeforeEeach / @AfterEeach
- @Test

④ Selection Sort test cases.

@Test

public void testSelectionSort ()

{

int [] actual = {5, 1, 4, 2, 8};

int [] expected = {1, 2, 4, 5, 8};

SelectionSort.sort (actual); // sorting method

assertEquals (expected, actual);

}

⑤ Example of assertTrue() & assertEquals()

@Test

public void testCalculate ()

{

int result = 5 + 5;

assertEquals (10, result); // check equals

assertTrue (result > 0); // check condition

}